

Programming Habits

Instructor: Mark Kramer

Good programming habits

In this course, we'll write/read programs:

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  I    = 1                #Set the parameter I.
5  C    = 1                #Set the parameter C.
6  dt   = 0.01             #Set the timestep.
7  V    = np.zeros([1000,1]) #Initialize V.
8  V[0] = 0.2;             #Set the initial value of V.
9
10 for k in range(1,999):   #March forward in time,
11     V[k+1] = V[k] + dt*(I/C) #... updating V along the way.
12
13 t = np.arange(0, len(V))*dt #Define the time axis.
14
15 plt.figure()
16 plt.plot(t,V)             #Plot the results.
17 plt.xlabel('Time [s]')
18 plt.ylabel('Voltage [mV]')
19 plt.show()
```

code

comments

Good programming habits

Q. What makes a program “good” ?

A.

- It works
- It’s interpretable
- It’s extendible

Write programs for people, not computers

You should think of (at least) 3 other people programing:

Person 1: Yourself, tomorrow.

Person 2: Yourself, in six months.

Person 3: Another user of your code.

Always code as if the [person] who ends up maintaining your code will be a violent psychopath who knows where you live.

— John Woods `comp.lang.c++`

Write programs for people, not computers

How? **Comment your code, thoroughly.**

- carefully commenting code will make you happier.
 - your future self
 - popular and appreciated in your research group.

<code>dt=0.1;</code>	<code>%Set the timestep.</code>
<code>V(1)=Vrest;</code>	<code>%Set the initial condition.</code>
 <code>n_spikes = 0;</code>	 <code>%The # of spikes detected.</code>
 <code>Vstar = Iinput*R + Vrest;</code>	 <code>%Define Vstar = target voltage.</code>
<code>tau = R*C;</code>	<code>%Define tau = time constant</code>

Write programs for people, not computers

How? **Make comments meaningful.**

Ex.

<code>i = 10</code>	<code>% i equals 10</code>	NO!
---------------------	----------------------------	------------

<code>plot(x,y)</code>	<code>% plot x versus y</code>	NO!
------------------------	--------------------------------	------------

Ex.

<code>i = 10</code>	<code>% Define the number of iterations</code>
---------------------	--

<code>plot(x,y)</code>	<code>% Plot impulse (x) vs rate (y)</code>
------------------------	---

Write programs for people, not computers

How? Use comments to break programs into logical blocks

Ex.

4+3

NO!

4/10^2

NO!

Ex.

```
%% Example 1: addition.
```

```
4+3
```

```
%% Example 2: division and powers.
```

```
4/10^2
```


Write programs for people, not computers

How? **Make names consistent, distinctive, and meaningful.**

Ex.

```
a = 10  
b = ...  
c = my_function_18v2(a,b)
```

NO!

Ex.

```
n_trials = 10  
lfp_rat = ...  
lfp_mean = compute_trial_mean(n_trials,lfp_rat)
```


Write programs for people, not computers

How? Utilize spacing to make code stylish.

Ex. **NO!**

```
Iinput=3;  
R=10;  
C=0.6;  
Vrest=-70;  
Vthreshold=-60;  
Vreset=-80;
```

Ex.

```
 Iinput  =  3;  
R = 10;  
C = 0.6;  
Vrest = -70;  
Vthreshold= -60;  
Vreset = -80;
```

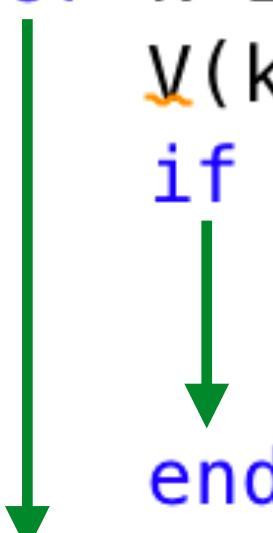
Write programs for people, not computers

How? Utilize spacing to make code stylish.

Ex. `for k=1:400-1`
`V(k+1) = V(k) + dt*(-(V(k)-Vstar)/tau);`
`if V(k+1) > Vth`
`V(k+1) = Vreset;`
`n_spikes = n_spikes+1;`
`end`
`end`

NO!

Ex. `for k=1:400-1`
`V(k+1) = V(k) + dt*(-(V(k)-Vstar)/tau);`
`if V(k+1) > Vth`
`V(k+1) = Vreset;`
`n_spikes = n_spikes+1;`
`end`
`end`

A diagram illustrating code indentation. A long green arrow points from the 'for' loop line down to the 'end' line. A shorter green arrow points from the 'if' line down to its corresponding 'end' line, showing the nested structure of the code.

Write programs for people, not computers

How? **Break up your program into manageable parts.**

Ex. Books organized into chapters, paragraphs.

- Etymology
- Extracts
 1. Loomings
 2. The Carpet-Bag
 3. The Spouter-Inn
 4. The Counterpane
 5. Breakfast
 6. The Street
 7. The Chapel
 8. The Pulpit

CHAPTER 1

LOOMINGS

Call me Ishmael. Some years ago — never mind how long precisely — having little or no money in my purse, and nothing particular to interest me on shore, I thought I would sail about a little and see the watery part of the world. It is a way I have of driving off the spleen and regulating the circulation. Whenever I find myself growing grim about the mouth; whenever it is a damp, drizzly November in my soul; whenever I find myself involuntarily pausing before coffin warehouses, and bringing up the rear of every funeral I meet; and especially whenever my hypos get such an upper hand of me, that it requires a strong moral principle to prevent me from deliberately stepping into the street, and methodically knocking people's hats off — then, I account it high time to get to sea as soon as I can. This is my substitute for pistol and ball. With a philosophical flourish Cato throws himself upon his sword; I quietly take to the ship. There is nothing surprising in this. If they but knew it, almost all men in their degree, some time or other, cherish very nearly the same feelings towards the ocean with me.

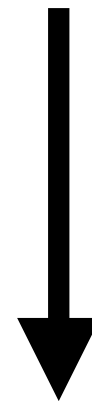
There now is your insular city of the Manhattoes, belted round by wharves as Indian isles by coral reefs — commerce surrounds it with her surf. Right and left, the streets take you waterward. Its extreme downtown is the battery, where that noble mole is washed by waves, and cooled by breezes, which a few hours previous were out of sight of land. Look at the crowds of water-gazers there.

Write programs for people, not computers

How? **Break up your program into manageable parts.**

Ex. Organize code into “pieces” - e.g. each piece a function.

```
d    = preprocessing(data)
pow  = power_spectrum(d)
coh  = coherence(d)
```



```
function output = preprocessing(d1)
```

```
function P = power_spectrum(d1)
```

```
function C = coherence(d1)
```

Write programs for people, not computers

How? **Break up your program into manageable parts.**

Ex. Organize code into “pieces” - e.g. each piece a function.

```
function output = preprocessing(data)
```

Maximum length of a function:

- ideally no more than one screen's worth of code.

Don't repeat yourself (or others).

How? **Make the computer repeat tasks**

Ex. Cut & paste over and over again.

```
a = [ [ some voltage trace ] ];
```

```
t = (0:1:length(a));
```

```
y = a.^2;
```

```
plot(t,y)
```

```
title([ 'Plot of voltage trace A' ])
```

... some data

... define axis for plot

... do something to data

... plot it

... with a title.

```
b = [ [ another voltage trace ] ];
```

```
t = (0:1:lengthb);
```

```
y = b.^2;
```

```
plot(t,y)
```

```
title([ 'Plot of voltage trace B' ])
```

... more data

... define axis for plot

... do something to data

... plot it

... with a title.

Don't repeat yourself (or others).

How? **Make the computer repeat tasks**

Ex. Cut & paste over and over again. Define a *function*

```
function square_and_plot(x, plot_title)
    t = (0:1:length(x));
    y = x.^2;
    plot(t,y)
    title(plot_title)
end
```

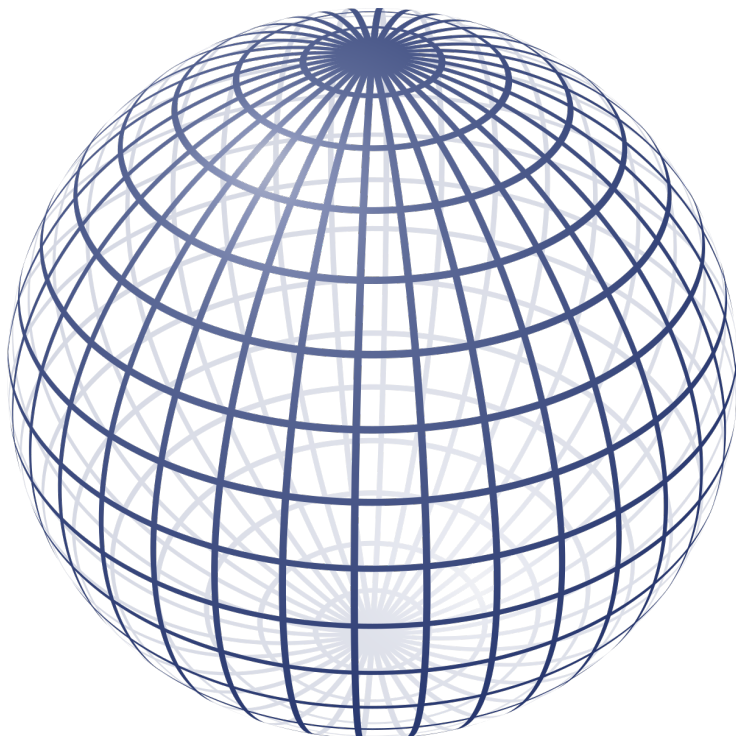
```
a = [ [ some voltage trace ] ];
b = [ [ another voltage trace ] ];
square_and_plot(a, 'Plot of voltage trace A')
square_and_plot(b, 'Plot of voltage trace B')
```


Don't repeat yourself (or others).

How? **Re-use existing functions (that you trust)**

It's usually better to find an established library or package that solves a problem than to attempt to write your own routines for well established problems.

Ex. Compute the distance between two points on a sphere.



MATLAB:

```
distance(lat1,lon1,lat2,lon2)
```

Python:

```
distance = geodesic(pt1,pt2).miles
```


Visualize data

How? **When in doubt, plot it out.**

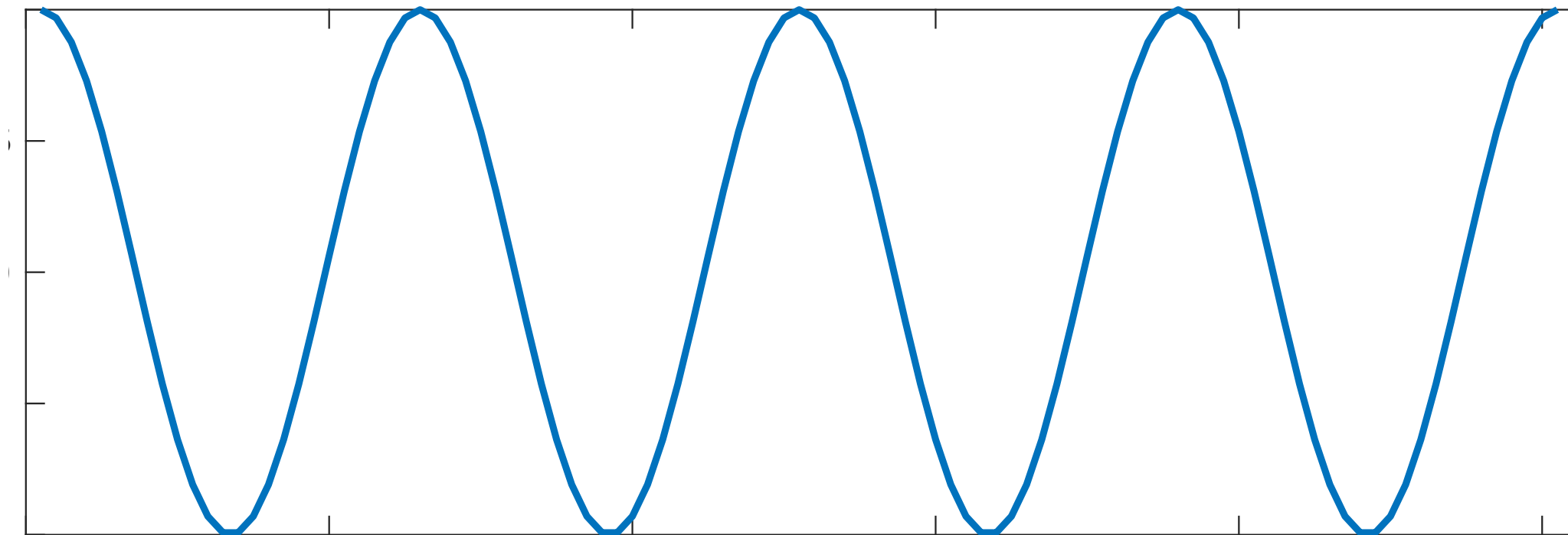
Ex. You're give the data in variable `x`. Let's look at it.

```
x =  
Columns 1 through 6  
    1.0000    0.9686    0.8763    0.7290    0.5358  
Columns 7 through 12  
    0.0628   -0.1874   -0.4258   -0.6374   -0.8090  
Columns 13 through 18  
   -0.9921   -0.9921   -0.9298   -0.8090   -0.6374  
Columns 19 through 24  
   -0.1874    0.0628    0.3090    0.5358    0.7290  
Columns 25 through 30  
    0.9686    1.0000    0.9686    0.8763    0.7290  
Columns 31 through 36  
    0.3090    0.0628   -0.1874   -0.4258   -0.6374  
Columns 37 through 42  
   -0.9298   -0.9921   -0.9921   -0.9298   -0.8090
```

Visualize data

How? **When in doubt, plot it out.**

Ex. You're give the data in variable **x**. Let's plot it.



Visualizing the data: the structure becomes clear.

Do something

Do not let these rules **stop** you from coding.

- The most important is to write code.
- Think back to these “rules” from time to time ...

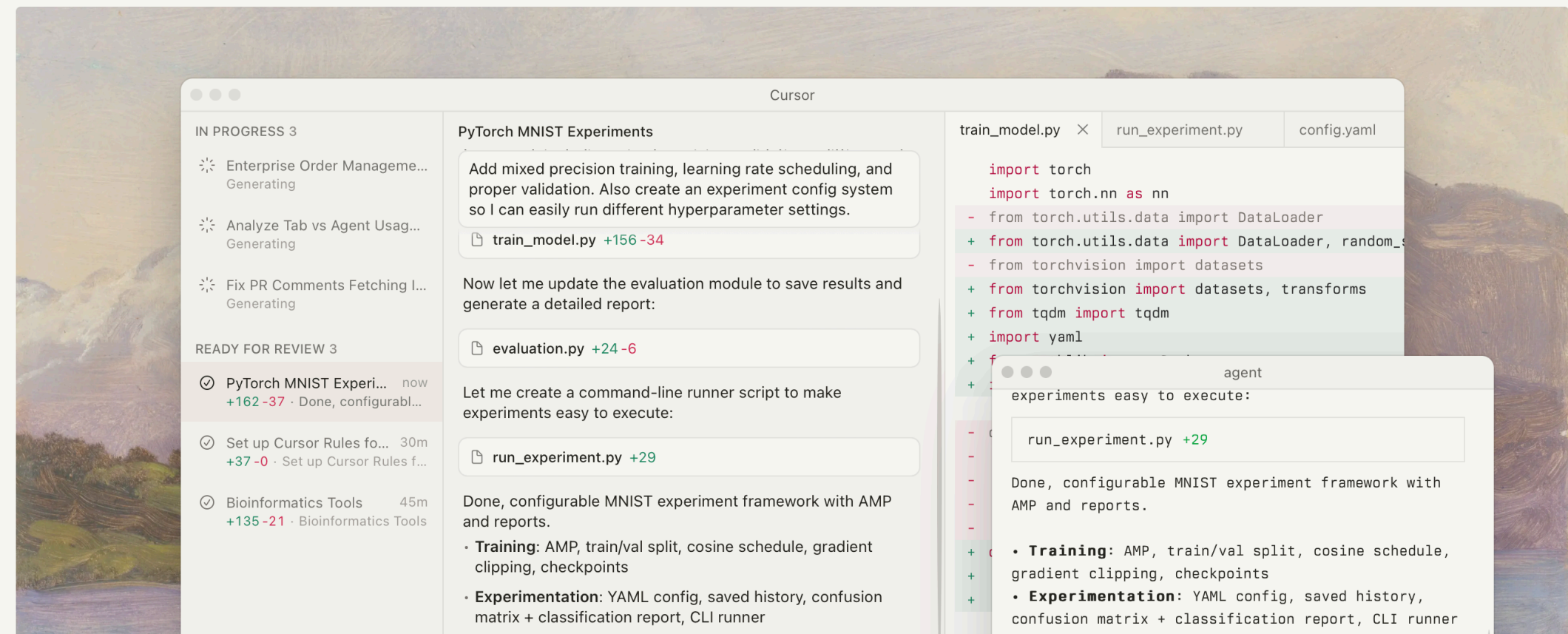
Some examples from here

<https://mark-kramer.github.io/Case-Studies-Python/01.html>

Who writes programs anymore?

[Features](#)[Enterprise](#)[Pricing](#)[Resources](#)[Sign in](#)[Download](#)

Built to make you extraordinarily productive,
Cursor is the best way to code with AI.

[Download for macOS ↓](#)

Who writes programs anymore?

Ex. Homework problem:

Determine how the firing rate of the HH model varies with input current I . Plot the firing rate versus I (i.e., plot the “ f - I curve”).

What you can do?

Oversee your LLM helper

- Ask the question
- Know the answer - in some sense
- Check the work

Can be a challenge

Could I clearly and thoroughly explain every step of this solution to a classmate, without relying on AI?

Summary

It's (still) important to read computer code.

Emerging practice: oversee the work of an LLM.

You ask the big questions, and check the LLM's work.

Could I clearly and thoroughly explain every step of this solution to a classmate, without relying on AI?